```
#Logistic Regression (Classification)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# data loading
dataset=pd.read_csv("Placement selection dataset.csv")
dataset

#one-hot encoding
data=pd.get_dummies(dataset,drop_first=True)

#data split input and output
independent=data[['Age', 'Internships', 'CGPA', 'Hostel', 'HistoryOfBacklogs', 'Gender_Male', 'Stream_Computer Science', 'St
dependent=data['PlacedOrNot']

#training and testing
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(
    independent,dependent,test_size=0.30,
    random_state=50,stratify=dependent
)

#Feature scaling
from sklearn.preprocessing import StandardScaler

sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)

#Model creation and hyperparameter tuning
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

model=LogisticRegression(max_iter=2000)

param_grid={
    'C':[0.01,0.1,1,10,100],
    'class_weight':[None,'balanced']
}

grid=GridSearchCV(
    model,param_grid,cv=5,
    scoring='accuracy',n_jobs=-1
)

grid.fit(x_train,y_train)

classifier=grid.best_estimator_

print("Best Parameters:",grid.best_params_)
print("Best Cross-Validation Accuracy:",grid.best_score_)

weight=classifier.coef_
print("Weights (Coefficients):",weight)

bias=classifier.intercept_
print("Bias (Intercept):",bias)

y_pred=classifier.predict(x_test)
y_pred

#Evaluation metrics
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

accuracy=accuracy_score(y_test,y_pred)
print('Accuracy Score:',accuracy)
print('Accuracy Percentage:',round(accuracy*100,2),'%')

print('Confusion Matrix:\n',confusion_matrix(y_test,y_pred))
print('\nClassification Report:\n',
      classification_report(y_test,y_pred,zero_division=0))

#Prediction
sample=pd.DataFrame(
    [[22,1,8,1,1,1,0,0,1,0,0]],
    columns=independent.columns
)
```

```
sample_scaled=sc.transform(sample)
predict=classifier.predict(sample_scaled)
print("Single Prediction (Class 0 or 1):",predict)

#deployment
import pickle

filename="placement_selection_model.sav"

#Save both scaler and classifier
pickle.dump(
    {'scaler':sc,'classifier':classifier},
    open(filename,'wb')
)

#Use in another file
loaded_model=pickle.load(open("placement_selection_model.sav",'rb'))

predict=loaded_model['classifier'].predict(
    loaded_model['scaler'].transform(sample)
)
print("Loaded Model Prediction:",predict)

# Get values from the user
age=int(input("Enter your Age: "))
internships=int(input("Enter number of Internships: "))
cgpa=float(input("Enter CGPA: "))
hostel=int(input("Enter Hostel (0 or 1): "))
history_of_backlogs=int(input("Enter History of Backlogs (0 or 1): "))
gender_male=int(input("Enter Gender (1 for Male, 0 for Female): "))
stream_cs=int(input("Enter Computer Science Stream (0 or 1): "))
stream_ee=int(input("Enter Electrical Stream (0 or 1): "))
stream_ece=int(input("Enter Electronics and Communication Stream (0 or 1): "))
stream_it=int(input("Enter Information Technology Stream (0 or 1): "))
stream_me=int(input("Enter Mechanical Stream (0 or 1): "))

#Pass user inputs to model prediction
user_data=pd.DataFrame(
    [[age,internships,cgpa,hostel,history_of_backlogs,
      gender_male,stream_cs,stream_ee,stream_ece,stream_it,stream_me]],
    columns=independent.columns
)

user_data_scaled=loaded_model['scaler'].transform(user_data)

predict=loaded_model['classifier'].predict(user_data_scaled)
print("Predicted Class for input user:",predict)
```

```
Best Parameters: {'C': 0.01, 'class_weight': 'balanced'}
Best Cross-Validation Accuracy: 0.6535714285714286
Weights (Coefficients): [[-0.00234387  0.01985969  0.08057484  0.00204584  0.0014843   0.01156316
  -0.04768048  0.00628312  0.00618838  0.03051883  0.03120592]]
Bias (Intercept): [0.01674804]
Accuracy Score: 0.8125
Accuracy Percentage: 81.25 %
Confusion Matrix:
 [[5 1]
 [2 8]]

Classification Report:
              precision    recall  f1-score   support

           0       0.71      0.83      0.77         6
           1       0.89      0.80      0.84        10

    accuracy                           0.81        16
   macro avg       0.80      0.82      0.81        16
weighted avg       0.82      0.81      0.81        16

Single Prediction (Class 0 or 1): [1]
Loaded Model Prediction: [1]
Enter your Age: 23
Enter number of Internships: 2
Enter CGPA: 8
Enter Hostel (0 or 1): 1
Enter History of Backlogs (0 or 1): 0
Enter Gender (1 for Male, 0 for Female): 0
Enter Computer Science Stream (0 or 1): 1
Enter Electrical Stream (0 or 1): 0
Enter Electronics and Communication Stream (0 or 1): 0
Enter Information Technology Stream (0 or 1): 0
Enter Mechanical Stream (0 or 1): 0
Predicted Class for input user: [1]
```

```
#K-Nearest Neighbors (Classification)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# data loading
dataset=pd.read_csv("Placement selection dataset.csv")
dataset

#one-hot encoding
data=pd.get_dummies(dataset,drop_first=True)

#data split input and output
independent=data[['Age', 'Internships', 'CGPA', 'Hostel', 'HistoryOfBacklogs', 'Gender_Male', 'Stream_Computer Science', 'S
dependent=data['PlacedOrNot']

#training and testing
from sklearn.model_selection import train_test_split

x_train,x_test,y_train,y_test=train_test_split(
    independent,dependent,test_size=0.30,
    random_state=50,stratify=dependent
)

#Feature scaling (without Pipeline)
from sklearn.preprocessing import StandardScaler

sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)

#Model creation and hyperparameter tuning
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

model=KNeighborsClassifier()

param_grid={
    'n_neighbors':[3,5,7,9,11,13,15],
    'weights':['uniform','distance'],
    'p':[1,2]
}

grid=GridSearchCV(
    model,param_grid,cv=5,
    scoring='accuracy',n_jobs=-1
)

grid.fit(x_train,y_train)

classifier=grid.best_estimator_

print("Best Parameters:",grid.best_params_)
print("Best Cross-Validation Accuracy:",grid.best_score_)

#Model prediction
y_pred=classifier.predict(x_test)
y_pred

#Evaluation metrics
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

accuracy=accuracy_score(y_test,y_pred)
print('Accuracy Score:',accuracy)
print('Accuracy Percentage:',round(accuracy*100,2),'%')
print('Confusion Matrix:\n',confusion_matrix(y_test,y_pred))
print('\nClassification Report:\n',
      classification_report(y_test,y_pred,zero_division=0))

#Prediction
sample=pd.DataFrame(
    [[22,1,8,1,1,1,0,0,1,0,0]],
    columns=independent.columns
)

sample_scaled=sc.transform(sample)
predict=classifier.predict(sample_scaled)
print("Single Prediction (Class 0 or 1):",predict)

#deployment
```

```
import pickle

filename="placement_selection_knn_model.sav"

pickle.dump(
    {'scaler':sc,'classifier':classifier},
    open(filename,'wb')
)

#Use in another file
loaded_model=pickle.load(
    open("placement_selection_knn_model.sav",'rb')
)

predict=loaded_model['classifier'].predict(
    loaded_model['scaler'].transform(sample)
)
print("Loaded Model Prediction:",predict)

# Get values from the user
age=int(input("Enter your Age: "))
internships=int(input("Enter number of Internships: "))
cgpa=float(input("Enter CGPA: "))
hostel=int(input("Enter Hostel (0 or 1): "))
history_of_backlogs=int(input("Enter History of Backlogs (0 or 1): "))
gender_male=int(input("Enter Gender (1 for Male, 0 for Female): "))
stream_cs=int(input("Enter Computer Science Stream (0 or 1): "))
stream_ee=int(input("Enter Electrical Stream (0 or 1): "))
stream_ece=int(input("Enter Electronics and Communication Stream (0 or 1): "))
stream_it=int(input("Enter Information Technology Stream (0 or 1): "))
stream_me=int(input("Enter Mechanical Stream (0 or 1): "))

#Pass user inputs to model prediction
user_data=pd.DataFrame(
    [[age,internships,cgpa,hostel,history_of_backlogs,
      gender_male,stream_cs,stream_ee,stream_ece,stream_it,stream_me]],
    columns=independent.columns
)

user_data_scaled=loaded_model['scaler'].transform(user_data)

predict=loaded_model['classifier'].predict(user_data_scaled)
print("Predicted Class for input user:",predict)
```

```
Best Parameters: {'n_neighbors': 9, 'p': 1, 'weights': 'uniform'}
Best Cross-Validation Accuracy: 0.7107142857142857
Accuracy Score: 0.75
Accuracy Percentage: 75.0 %
Confusion Matrix:
 [[4 2]
 [2 8]]

Classification Report:
              precision    recall  f1-score   support

           0       0.67      0.67      0.67         6
           1       0.80      0.80      0.80        10

    accuracy                           0.75        16
   macro avg       0.73      0.73      0.73        16
weighted avg       0.75      0.75      0.75        16

Single Prediction (Class 0 or 1): [1]
Loaded Model Prediction: [1]
Enter your Age: 27
Enter number of Internships: 3
Enter CGPA: 9
Enter Hostel (0 or 1): 1
Enter History of Backlogs (0 or 1): 1
Enter Gender (1 for Male, 0 for Female): 0
Enter Computer Science Stream (0 or 1): 0
Enter Electrical Stream (0 or 1): 1
Enter Electronics and Communication Stream (0 or 1): 0
Enter Information Technology Stream (0 or 1): 0
Enter Mechanical Stream (0 or 1): 0
Predicted Class for input user: [1]
```

```
#Naive Bayes (Classification)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
# data loading
dataset=pd.read_csv("Placement selection dataset.csv")
dataset

#one-hot encoding
data=pd.get_dummies(dataset,drop_first=True)

#data split input and output
independent=data[['Age', 'Internships', 'CGPA', 'Hostel', 'HistoryOfBacklogs', 'Gender_Male', 'Stream_Computer Science', 'St
dependent=data['PlacedOrNot']

#training and testing
from sklearn.model_selection import train_test_split

x_train,x_test,y_train,y_test=train_test_split(
    independent,dependent,test_size=0.30,
    random_state=50,stratify=dependent
)

#Model creation and hyperparameter tuning
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import GridSearchCV

model=GaussianNB()

param_grid={
    'var_smoothing':[1e-12,1e-11,1e-10,1e-9,1e-8,1e-7,1e-6]
}

grid=GridSearchCV(
    model,param_grid,cv=5,
    scoring='accuracy',n_jobs=-1
)

grid.fit(x_train,y_train)

classifier=grid.best_estimator_

print("Best Parameters:",grid.best_params_)
print("Best Cross-Validation Accuracy:",grid.best_score_)

#Model prediction
y_pred=classifier.predict(x_test)
y_pred

#Evaluation metrics
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

accuracy=accuracy_score(y_test,y_pred)
print('Accuracy Score:',accuracy)
print('Accuracy Percentage:',round(accuracy*100,2),'%')

print('Confusion Matrix:\n',confusion_matrix(y_test,y_pred))
print('\nClassification Report:\n',
      classification_report(y_test,y_pred,zero_division=0))

#Prediction
sample=pd.DataFrame(
    [[22,1,8,1,1,1,0,0,1,0,0]],
    columns=independent.columns
)

predict=classifier.predict(sample)
print("Single Prediction (Class 0 or 1):",predict)

#deployment
import pickle

filename="placement_selection_nb_model.sav"
pickle.dump(classifier,open(filename,'wb'))

#Use in another file
loaded_model=pickle.load(
    open("placement_selection_nb_model.sav",'rb')
)

predict=loaded_model.predict(sample)
print("Loaded Model Prediction:",predict)

# Get values from the user
age=int(input("Enter your Age: "))
internships=int(input("Enter number of Internships: "))
cgpa=float(input("Enter CGPA: "))
```

```
cgpa=float(input("Enter CGPA: "))
hostel=int(input("Enter Hostel (0 or 1): "))
history_of_backlogs=int(input("Enter History of Backlogs (0 or 1): "))
gender_male=int(input("Enter Gender (1 for Male, 0 for Female): "))
stream_cs=int(input("Enter Computer Science Stream (0 or 1): "))
stream_ee=int(input("Enter Electrical Stream (0 or 1): "))
stream_ece=int(input("Enter Electronics and Communication Stream (0 or 1): "))
stream_it=int(input("Enter Information Technology Stream (0 or 1): "))
stream_me=int(input("Enter Mechanical Stream (0 or 1): "))

#Pass user inputs to model prediction
user_data=pd.DataFrame(
    [[age,internships,cgpa,hostel,history_of_backlogs,
      gender_male,stream_cs,stream_ee,stream_ece,stream_it,stream_me]],
    columns=independent.columns
)

predict=loaded_model.predict(user_data)
print("Predicted Class for input user:",predict)
```

```
Best Parameters: {'var_smoothing': 1e-07}
Best Cross-Validation Accuracy: 0.5178571428571429
Accuracy Score: 0.4375
Accuracy Percentage: 43.75 %
Confusion Matrix:
 [[6 0]
 [9 1]]

Classification Report:
              precision    recall  f1-score   support

           0       0.40      1.00      0.57         6
           1       1.00      0.10      0.18        10

    accuracy                           0.44        16
   macro avg       0.70      0.55      0.38        16
weighted avg       0.78      0.44      0.33        16

Single Prediction (Class 0 or 1): [0]
Loaded Model Prediction: [0]
Enter your Age: 45
Enter number of Internships: 0
Enter CGPA: 5
Enter Hostel (0 or 1): 0
Enter History of Backlogs (0 or 1): 1
Enter Gender (1 for Male, 0 for Female): 1
Enter Computer Science Stream (0 or 1): 1
Enter Electrical Stream (0 or 1): 0
Enter Electronics and Communication Stream (0 or 1): 0
Enter Information Technology Stream (0 or 1): 0
Enter Mechanical Stream (0 or 1): 0
Predicted Class for input user: [0]
```